PYTHON PROGRAMMING – ASSIGNMENT 2

CO2:ControlFlow,StringOperationsandFunctions

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences, Chennai

Student Name: Rakshith.S

Register Number: 1912260391

Course Code: CSA0805

Assignment: Assignment 2 – CO2

Total Marks: 50

Submission Date: 30.10.2026

GitHub Repository: CSA0805-1912260391-Assignment2

GitHub Link: https://github.com/Rakshith-SIMATS/CSA0805-1912260391-Assignment2.git

Contents

• Task 1 – Decisions with Many Boundaries

• Task 2 – Loops that Count Attempts

• Task 3 – Vehicle Registration Number Validator

• Task 4 – Building Long Text: + or join()?

• Task 5 – Functions, Parameters and Scope

• Task 6 – Recursion and Its Limits

• Task 7 – Diagnose, Redesign and Validate

• Submission Checklist

Task 1 – Decisions with Many Boundaries

Q1 – Decisions with Many Boundaries (6 Marks)

Bloom'sLevel:K4– Analyse

Problem context

A household receives a full subsidy when consumption is 100 units or less. For 101–200 units, it receives a partial subsidy only when it has a ration card. Above 200 units, no subsidy is given unless a household member is older than 80; for this assignment, that exception is called a special subsidy.

Predictions before execution

Units	Ration card	Member > 80	Prediction
100	No	No	Full subsidy
101	Yes	No	Partial subsidy
101	No	No	No subsidy
200	Yes	No	Partial subsidy
200	No	No	No subsidy
201	No	Yes	Special subsidy
201	Yes	No	No subsidy
50	Yes	Yes	Full subsidy

Implementation 1 – separate if statements

```
def separate_if(units,ration, elderly):
    # These are independent checks. Overlapping conditions can print
# more than one result, which is the bug this version demonstrates.
    if units <= 200:
        print("At or below 200 units")

    if units <= 100:
        print("Full subsidy")

    if 101 <= units <= 200 and ration:
        print("Partial subsidy")

    if 101 <= units <= 200 and not ration:
        print("No subsidy")

    if units > 200 and elderly:
        print("Special subsidy")

    if units > 200 and not elderly:
        print("No subsidy")
```

Implementation 2 – if-elif-else

```
def if_elif_else(units,ration,elderly):
    if units <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200:
        if ration:
            return "Partial subsidy"
        else:
            return "No subsidy"
    else:
        if elderly:
            return "Special subsidy"
        else:
            return "No subsidy"



households = [
    (100, False, False), (101, True, False),
    (101, False, False), (200, True, False),
    (200, False, False), (201, False, True),
    (201, True, False), (50, True, True)
]

for household in households:
    print(household,"->",if_elif_else(*household))
```

Expected chain results

```
(100, False, False) -> Full subsidy
(101, True, False) -> Partial subsidy
(101, False, False) -> No subsidy
(200, True, False) -> Partial subsidy
(200, False, False) -> No subsidy
(201, False, True) -> Special subsidy
(201, True, False) -> No subsidy
(50, True, True)    -> Full subsidy
```

Separate if statements each run independently, so overlapping conditions can produce multiple messages. An if–elif–else chain selects only the first matching branch. The chain returns exactly one decision for these cases.

Design decision

Use if–elif–else with a nested if for the additional eligibility checks. The consumption ranges are mutually exclusive, and the nested checks make the ration-card and elderly-member requirements clear.

Alternative concept and complexity

A match–case solution can first classify consumption into a range and then match the range and eligibility flags. A lookup list of (upper_limit, decision) pairs can be searched until the first matching limit is found. Conditional branches take O(1) time for a fixed number of slabs and O(1) auxiliary space. A sequential lookup takes O(k) time for k slabs and O(k) space for the table.

Task 2 – Loops that Count Attempts

Q2 – Loops that Count Attempts (4 Marks)

Bloom'sLevel:K3–Apply

Implementation using while

```
correct_otp = "1234"
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    otp = input("Enter OTP: ").strip()

    if otp == "":
        print("Blank entry does not count. Try again.")
        continue

    attempts += 1

    if otp == correct_otp:
        print("OTP verified successfully")
        break

    print("Wrong OTP")

if attempts == 2:
        pass # Placeholder: a warning could be added here.
else:
    print("Accountlocked for30minutes")
```

Implementation using for and range(3)

To ensure blank entries do not consume one of the three attempts, each counted attempt can contain a small input loop that repeats until a non-blank entry is supplied.

```
correct_otp = "1234"

for attempt in range(3):
    otp = input("Enter OTP: ").strip()
    while otp == "":
        print("Blank entry does not count. Try again.")
        otp = input("Enter OTP: ").strip()

    if otp == correct_otp:
        print("OTP verified successfully")
        break

    print("Wrong OTP")
else:
    print("Account locked for 30 minutes")
```

Demonstration of loop controls

break exits the loop when the OTP is correct. continue skips the rest of the current while-loop iteration for a blank entry. pass is a no-operation placeholder. The loop else clause runs only if the loop ends normally without break, so it is suitable for the lock message.

Changed condition: two extra attempts

If the security question is answered correctly, set max_attempts = 5; otherwise set it to 3. Use the while-loop version with attempts < max_attempts. For the for-loop version, use range(max_attempts) and keep the blank-entry input loop inside each counted attempt.

Recursive alternative and complexity

A recursive function could accept the number of attempts remaining and call itself after a wrong OTP. All approaches take O(a) time for a attempts. A loop uses O(1) auxiliary space; recursion uses O(a) call-stack space. A loop is preferable for this small, bounded verification process. Updating the counter before checking for a blank input incorrectly counts blank entries; failing to update it after a wrong OTP may allow unlimited attempts.

Task 3 – Vehicle Registration Number Validator

Q3–EngineeraVehicleRegistrationNumberValidator(6 Marks)

Bloom's Level: K4 – Analyse

Predictions before execution

Raw input	Normalised form	Prediction
tn 09 ab 1234	TN09AB1234	Valid Valid
KA-01-M-0007	KA01M0007	Invalid
TN9AB1234	TN9AB1234	Invalid
TN 22 1234 AB	TN221234AB	Invalid
mh 12 xyz 6789	MH12XYZ6789	Valid
KL 07 P 4567	KL07P4567	

Completed code

```
def normalise(text):
 """Remove spaces and hyphens, then convert to uppercase."""
    text = text.strip().upper()
    text = text.replace(" ", "")
    text = text.replace("-", "")
    return text


def is_valid(reg):
    """Check the pattern: 2 letters, 2 digits, 1–2 letters, 4 digits."""
    if len(reg) not in (9, 10):
        return False
```

```
    middle = reg[4:-4]
    return (
        reg[0:2].isalpha()
        and reg[2:4].isdigit()
        and len(middle) in (1, 2)
        and middle.isalpha()
        and reg[-4:].isdigit()
    )


def format_receipt(reg):
 """Format a valid registration number for a receipt."""
    if not is_valid(reg):
        return "Invalid registration number"
    return reg[:2] + " " + reg[2:4] + " " + reg[4:-4] + " " + reg[-4:]


inputs = [
 "tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234",
"TN 22 1234 AB", "mh 12 xyz 6789", "KL 07 P 4567"
]

for raw in inputs:
 reg = normalise(raw)
    status = "valid" if is_valid(reg) else "INVALID"
 print(f"{raw:18} -> {reg:11} {status}")

print(format_receipt(normalise("tn 09 ab 1234")))
```

Expected output

```
tn 09 ab 1234      -> TN09AB1234 valid
KA-01-M-0007       -> KA01M0007 valid
TN9AB1234          -> TN9AB1234 INVALID
TN 22 1234 AB      -> TN221234AB INVALID
mh 12 xyz 6789     -> MH12XYZ6789 INVALID
KL 07 P 4567       -> KL07P4567 valid
TN 09 AB 1234
```

String immutability and slicing

Python strings are immutable: their characters cannot be changed in place. text.upper() returns a new uppercase string and does not modify text; assign the result if you want to keep it. For a 9-character string, s[-4:] returns the last four characters and s[4:-4] returns one character. For a 10-character string, s[-4:] returns the last four characters and s[4:-4] returns two characters.

Regular-expression alternative and complexity

```
import re

def is_valid_regex(reg):
return bool(re.fullmatch(
```

```
    r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg
))
```

Both slicing-based validation and fullmatch are O(L) time for a string of length L. A regular expression is concise for complex patterns; slicing makes each field boundary explicit. This assignment's primary implementation uses only string methods and slicing.

Task 4 – Building Long Text: + or join()?

Q4–BuildingLongText: +orjoin()?(6Marks)

Bloom's Level: K4 – Analyse

Prediction

join() is expected to perform well when combining many strings because it can allocate the result efficiently. Repeated += may require repeated copying, although CPython can optimise some cases. Timings depend on the machine and Python version.

Three implementations and benchmark

```
items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]
def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text
def build_join(parts):
    return "".join(p + ";" for p in parts)
def build_fstring(parts):
    return "".join(f"{p};" for p in parts)
for n in (1_000, 10_000, 200_000):
data = items[:n]
    a = build_plus(data)
    b = build_join(data)
    c = build_fstring(data)
    print(f"Lines: {n}; all outputs identical: {a == b == c}")
    print("Output characters:", len(a))
print("Benchmark timings omitted as requested.")
```

How to interpret the measurements

Benchmark timings are omitted as requested. The execution screenshot confirms that all three methods create identical output at 1,000, 10,000 and 200,000 lines.

Alternative: file output or StringIO

```
# Write directly to a file without retaining the whole statement:
with open("statement.txt", "w", encoding="utf-8") as file:
 for item in items:
        file.write(item + ";")
```

```
# Or build an in-memory text buffer:
from io import StringIO

buffer = StringIO()
for item in items:
 buffer.write(item + ";")
statement = buffer.getvalue()
```

Let N be the total number of output characters. join() is O(N) time and needs memory for the parts and final result. Repeated concatenation can copy up to $O(N^2)$ characters in the worst case. Streaming to a file uses a small buffer but does not keep the entire result in memory. StringIO keeps the accumulated output in memory.

Task 5 – Functions, Parameters and Scope

Q5– Functions, Parameters and Scope (7Marks)

Bloom's Level: K4 – Analyse

Completedcode

```
def add_item_buggy(item, order=[]):
order.append(item)
    return order


def add_item(item, order=None):
    if order is None:
        order = []
 order.append(item)
    return order


print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"), add_item("Vada"))
print("Defaults:", add_item_buggy.__defaults__)


def calculate_bill(price, quantity=1, gst_rate=0.05):
    subtotal = price * quantity
    gst = subtotal * gst_rate
    total = subtotal + gst
    return total, gst


total, gst = calculate_bill(100, quantity=2, gst_rate=0.05)
print("Total:", total)
print("GST:", gst)


college = "ABC College"
```

```
def show_college():
 college_name = "Engineering College"
print("Local:", college_name)
    print("Global:", college)

show_college()

count = 0
def increase():
    global count
    count += 1

increase()
print("Global count:", count)



def outer():
    value = 10
    def inner():
        nonlocal value
        value += 5
    inner()
    print("Nonlocal value:", value)

outer()
```

Expected output

```
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']
Defaults: (['Tea', 'Vada'],)
Total: 210.0
GST: 10.0
Local: Engineering College
Global: ABC College
Global count: 1
Nonlocal value: 15
```

Explanation

A defaultvalue is created once when a function is defined. Since the default list is mutable, calls to add_item_buggy share the same list. Using None as the default and creating a new list inside the function avoids the bug.

Positional arguments are matched by position, keyword arguments by parameter name, and default parameters supply values when omitted. A function can return multiple values as a tuple, such as (total, gst).

Local names exist within a function. Global names are defined at module level. nonlocal refers to a name in an enclosing function. The global keyword is required to rebind a module-level name from inside a function; excessive global state makes programs harder to test and maintain.

Explicit-list and closure alternatives

```
#Explicitlypass a separatelistforeachcustomer
order1 = []
add_item("Tea", order1)
order2 = []
add_item("Vada", order2)

# A closure can keep a private list for each customer
def create_order():
    order = []
 def add(item):
        order.append(item)
        return order
    return add

customer1 = create_order()
customer2 = create_order()
print(customer1("Tea"))
print(customer2("Vada"))
```

Appending one item to a list is O(1) amortised time. An order containing m items uses O(m) storage. Explicit lists make ownership clear; closures provide private state for each customer.

Task 6 – Recursion and Its Limits

Q6 – Recursion and Its Limits (8 Marks)

Bloom'sLevel:K5–Evaluate

Completed recursive and memoised program

```
from functools import lru_cache

import sys

calls = 0

def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
 return ways_naive(n - 1) + ways_naive(n - 2)



@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
 return ways_memo(n - 1) + ways_memo(n - 2)
```

```
for n in range(1, 7):
 ways_memo.cache_clear()
 print(n, ways_memo(n))

calls = 0
print("naive:", ways_naive(25), "calls =", calls)

ways_memo.cache_clear()
calls = 0
print("memo :", ways_memo(25), "calls =", calls)

print("Recursion limit:", sys.getrecursionlimit()) try:
 ways_memo.cache_clear()
  print("Ways  for  1000  steps:",  ways_memo(1000))
except RecursionError:


   print("RecursionError: recursion depth exceeded")



def ways_loop(n):
   if n <= 1:
      return 1
   a, b = 1, 1
   for _ in range(2, n + 1):
      a, b = b, a + b
   return b

print("Loop result for 1000 steps:", ways_loop(1000))
```

Base case and recursive case

Thebase casesare W(0) =1andW(1) = 1. For n greater than 1, W(n) = W(n−1) + W(n−2), because the first movecan beoneor two steps.

Call tree for n = 5

```
ways(5)
 ├─── ways(4)
 │   ├─── ways(3)
 │        |        ├──
ways(2)      |       |
 └─── ways(1)   |
 └──        ways(2)
 └─── ways(3)
   ├──    ways(2)
 └─── ways(1)
```

Hand-checked results

n	Ways
1	1
2	2

3	3
4	5
5	8
6	1
	3

Callcountsandrecursion limit

With an empty cache, the naive function makes 242,785 calls for n = 25 and returns 121,393. The memoised function should make 26 function-body calls for n = 25 when the cache is cleared first. The memoised call for n = 1,000 may raise RecursionError because recursive depth approaches the interpreter's recursion limit; sys.getrecursionlimit() reports the configured limit.

Iterative solution and complexity

The loop keeps only the previous two values. Plain recursion takes $O(\varphi^n)$ time and O(n) stack space. Memoisation takes O(n) time and O(n) space for its cache and recursion stack. The iterative version takes O(n) time and O(1) auxiliary space. The result itself is a very large integer, so its storage still grows with n.

Growth investigation

Measure call counts for n = 10, 20, 25 and 30 by resetting calls before each run. Clear the memoisation cache before each memoised measurement. Do not attempt plain recursion for n = 1,000; it is computationally impractical. Use the iterative method for deployment.

Task 7 – Diagnose, Redesign and Validate

Q7–Diagnose, RedesignandValidate(13Marks)

Bloom's Level: K5 – Evaluate

A. Diagnose the bottleneck (2 marks)

Select the stair-climbing program from Q6. The naive recursive function repeats the same subproblems many times. For n = 25, it makes 242,785 calls to return 121,393. The number of calls grows exponentially with n.

B. Define the design objective (2 marks)

Calculate the same result for n = 1,000 without RecursionError, reduce the time complexity to O(n), and use O(1) auxiliary space.

C. Design the revised solution (4 marks)

The revised algorithm stores only the previous two results. It avoids repeated subproblems and does not create a recursive call stack.

```
def ways_original(n):
    if n <= 1:
        return 1
    return ways_original(n-1) + ways_original(n-2)
def ways_redesigned(n):
    if n <= 1:
```

```
        return 1
    previous,current = 1,1
    for _ in range(2,n+1):
        previous,current = current,previous+current
    return current
for n in (10,20,25,30):
    original_result = ways_original(n)
    redesigned_result = ways_redesigned(n)
    print("n =", n)
    print("Original result:", original_result)
    print("Redesigned result:", redesigned_result)
 print("Same answer:", original_result == redesigned_result)
print("n = 1000, redesigned result:", ways_redesigned(1000))
print("Benchmark timings omitted as requested.")
```

D. Predict before implementing (1 mark)

The redesigned program should return the same result and should be much faster as n grows, because it uses a linear loop rather than exponential recursive calls.

E. Implement and validate (2 marks)

Run both versions with n = 10, 20, 25 and 30. Compare results and measured times. Use the same computer and conditions for each test. For n = 1,000, test the redesigned version; do not run the naive version because it is impractical. Benchmark timings are omitted as requested; compare the returned answers instead.

Input	Original result/time	Redesigned result/time	Same result?
10	Record actual output	Record actual output	Yes/No
20	Record actual output	Record actual output	Yes/No
25	Record actual output	Record actual output	Yes/No
30	Record actual output	Record actual output	Yes/No
1000	Not practical for naive recursion	Record result/time	N/A

F.Defendthefinaldesign (2 marks)

The redesign meets the objective if it returns the same answers, completes n = 1,000 without a recursion-depth error, and uses linear time with constant auxiliary space. The trade-off is that the iterative version is less directly similar to the mathematical recurrence, but it is more suitable for large inputs. The recursive version remains useful for teaching recursion on small values.

Submission Checklist

Submission Checklist

Before submitting, complete the following items:

- ☐ Student name and register number have been added to the cover page.
- ☐ The GitHub repository link has been added to the cover page.

- ☐ Repository: CSA0805-1912260391-Assignment2.
- ☐ Create one folder per task (Task01 through Task07) with Python source files and output screenshots. Include
- ☐ README.md with student name, register number, Python version, libraries and versions, run commands, and a short result note for every task.
- ☐ Include requirements.txt. These programs use only the Python standard library; no third-party packages are required.
- ☐ Commit progressively, at least one commit per task.
- ☐ Student verification note: This report is a structured draft based on the assignment questions supplied. The programs were executed with sample inputs and the resulting console outputs are included as screenshots. Benchmark timings have been omitted as requested. Review the code and be prepared to explain it during evaluation.
- ☐ Check that code, output screenshots, calculations and GitHub link are correct.

Suggested README.md

```
# Python Programming –Assignment 2 (CO2)

- Student name: [Enter your name]
- Register number: [Enter your register number]
- Python version: [Record output of python –version]
- Libraries: Python standard library only

## Run commands
python Task01/q1.py
python Task02/q2.py
python Task03/q3.py
python Task04/q4.py
python Task05/q5.py
python Task06/q6.py
python Task07/q7.py

## Task results
- Task 1: Compare separate if statements with if–elif–else.
- Task 2: Validate OTP attempts and demonstrate loop controls.
- Task 3: Normalise, validate and format registration numbers.
- Task 4: Compare text-building methods using measured timings.
- Task 5: Demonstrate safe defaults, return values and scope.
- Task 6: Compare naive recursion, memoisation and iteration.
- Task 7: Validate an iterative redesign against the original.


GitHub repository: [PasterepositoryURLhere]
```

Student verification note: This report is a structured draft based on the assignment questions supplied. The programs were executed with sample inputs and the resulting console outputs are included as screenshots. Benchmark timings have been omitted as requested. Review the code and be prepared to explain it during evaluation.

Program Execution Screenshots

The following terminal-style screenshots weregeneratedfromthe actualexecution output of the assignment programs using sample inputs. Benchmark timingsareintentionally notshown.

Task 1



Task 2



Task 3



Task 4



Task 5



Task 6



Task 7

